

Table des matières

Labo 01 — Le premier regard sur les sources de Boréal	1
Avant de commencer	2
Étape 1 — Créer le dépôt 01_de et son .gitignore	2
Étape 2 — Compter, sonder et peser les fichiers reçus sans en charger un seul	3
Étape 3 — Mesurer ce que coûte un type deviné	6
Récapitulatif	7
Dépannage	8
Annexe — Tout le code, d'un seul tenant	8

Labo 01 — Le premier regard sur les sources de Boréal

Labo · Module 1/12 — Formation Data Engineering

Ce que vous allez faire. Créer votre dépôt de code, 01_de. Jeter un premier regard en SQL sur ce que Boréal livre, sans rien charger : compter, sonder, peser. Puis constater ce que coûte un type deviné, sur l'ERP et sur la télémétrie. Objectif de fin : un dépôt 01_de avec son .gitignore, une requête qui compte les 275 436 lignes de l'historien sans rien charger, le poids des deux sources que le Cours 02 rangera, et trois pièges vus de vos yeux : 20251232, un décalage perdu, une colonne json.

Deux terminaux servent dans ce labo, et chaque étape dit lequel. L'Anaconda Prompt, environnement `formations` actif, sert à Python, à Git et aux commandes du système. La ligne de commande DuckDB, qui s'ouvre par `duckdb` et se quitte par `.exit`, sert au SQL.

La liste de contrôle du cours :

- ☐ Socle du guide d'installation en place, mise en situation lue
- ☐ Dépôt Git 01_de créé dans `Projet_Boreal`, `.gitignore` validé, `git status` propre
- ☐ Premières requêtes DuckDB : 275 436 lignes d'historien, 26 538 lignes d'en-têtes de commande, 158 146 messages
- ☐ Les sources pesées : 13 725 380 octets d'historien, 70 570 165 de télémétrie, dont 41 fichiers vides
- ☐ Les pièges des types devinés vus : 20251232, un décalage perdu, une colonne json

Avant de commencer

Ce cours ne demande que le socle du parcours : Python, Git, DuckDB, et les données de Boréal dans `Projet_Boreal/sources/`. Aucun service à allumer : la base de l'ERP et Airflow arrivent aux Cours 04 et 06. Trois vérifications, dans l'Anaconda Prompt depuis `Data_Domain` :

```
conda activate formations
duckdb --version
dir Projet_Boreal\sources
```

L'invite affiche (`formations`), `duckdb` répond `v1.5.5` ou une version plus récente, et `sources` montre `LISEZMOI.md`, `dictionnaire_boreal.md`, `empreintes.sha256` et six dossiers : `corpus`, `erp`, `garanties`, `meteo`, `telemetrie`, `usine`. Si l'un des trois manque, le socle se monte en une à deux heures avec `Data_Domain/GUIDE_INSTALLATION.md`, une seule fois pour tout le parcours.

Et une lecture, avant la première commande : `Cours/Mise_en_situation_Boreal.md`. Elle dit pour qui vous travaillez, ce que Boréal attend de son ingénieur de données et ce que vous livrez à chaque cours. Dix minutes, et tout le reste de la formation s'y rapporte.

Étape 1 — Créer le dépôt `01_de` et son `.gitignore`

Théorie : `Theorie_01.md`, section 4.

Le travail commence par un dépôt pour le code. Ouvrez l'Anaconda Prompt, activez `formations`, placez-vous dans `Projet_Boreal`, puis :

```
mkdir 01_de
cd 01_de
git init
```

Deux dossiers, deux rôles. `Projet_Boreal/` est la plateforme de données de Boréal : les sources que ses systèmes livrent, ses services, puis le lac et l'entrepôt que vous bâtirez. Il n'est pas versionné, et tout ce qu'il contient se reconstruit. `01_de/` est votre dépôt Git, qui ne contiendra que du code. C'est l'organisation d'une vraie équipe : le code d'ingestion vit dans son dépôt, la plateforme est partagée, et chaque formation du parcours y ajoutera le sien. Cours après cours, `01_de` deviendra votre pipeline complet, puis la pièce de portfolio du Cours 12.

Un ingénieur de données ne versionne ni données brutes ni secrets. Les données, c'est réglé par construction : `sources/` vit à côté de votre dépôt, pas dedans. Reste à fermer la porte aux secrets et aux fichiers que Python fabrique. Toujours dans `01_de`, créez le fichier `.gitignore` :

```
notepad .gitignore
```

Le Bloc-notes propose de créer le fichier : acceptez. Collez-y ces lignes, puis enregistrez et fermez :

```
# Secrets (mots de passe, jetons) : jamais dans Git
.env

# Fichiers compilés par Python
__pycache__/
```

La première entrée ne protège encore rien. Au Cours 04, la connexion à la base de l'ERP demandera un mot de passe, qui aura besoin d'une place hors du dépôt : autant la prévoir.

Validez ce premier jalon, puis revenez dans `Projet_Boreal`, d'où partent les labos :

```
git add .gitignore
git commit -m "Cours 01 : le dépôt 01_de et son .gitignore"
git status
cd ..
```

`git status` répond que l'arbre de travail est propre (*nothing to commit, working tree clean*).

(Le message *warning: ... LF will be replaced by CRLF est normal sous Windows : Git normalise les fins de ligne, sans conséquence.*)

Étape 2 — Compter, sonder et peser les fichiers reçus sans en charger un seul

Théorie : Theorie_01.md, sections 5 et 6.

Depuis `Projet_Boreal`, ouvrez la ligne de commande DuckDB :

```
duckdb
```

DuckDB annonce qu'il est branché sur une base en mémoire, puis affiche son invite, `D`. Rien ne s'écrit sur le disque : les fichiers restent où ils sont. Une requête peut courir sur plusieurs lignes ; elle part quand vous tapez son point-virgule, puis Entrée.

Au-dessus de chaque requête, un commentaire (les lignes qui commencent par `--`) dit ce qu'elle doit rendre et, au besoin, ce que veut dire un autre résultat. DuckDB ignore ces lignes : collez-les avec la requête, ou laissez-les de côté.

```
-- Attendu : 275 436.
-- Un autre nombre : Le générateur a tourné avec --historien complet ou --jusqua, ou un fichier s'est ajouté à historien/.
SELECT COUNT(*) AS lignes
FROM 'sources/usine/historien/*.csv';
```

DuckDB répond 275 436. Il vient de lire les 92 fichiers d'un coup, grâce au motif `/*.csv`, sans rien charger dans une base : il interroge les fichiers sur place.

Regardez ensuite la structure qu'il a devinée :

```
-- Attendu : 4 colonnes. TagName varchar, Timestamp timestamp, Value double, Quality bigint.
DESCRIBE SELECT * FROM 'sources/usine/historien/*.csv';
```

DuckDB répond par un encadré qui porte le motif des fichiers en titre, puis une ligne par colonne, avec le type qu'il lui a donné. Quatre colonnes : `TagName` en texte (`varchar`), `Timestamp` en horodatage (`timestamp`), `Value` en nombre à virgule (`double`) et `Quality` en entier (`bigint`). La ligne de commande écrit les

types en minuscules ; SQL ne fait pas la différence, et la documentation les écrit souvent en majuscules (VARCHAR). Un CSV ne transporte aucun type : DuckDB les a devinés en lisant un échantillon. Ça tombe juste ici ; l'étape 3 montre pourquoi on ne peut pas compter là-dessus.

La répartition des codes de qualité :

```
-- Attendu : 3 lignes, 192 (275 080 lignes), 64 (35) et 0 (321), soit 275 436 au total.
```

```
SELECT Quality, COUNT(*) AS points
FROM 'sources/usine/historien/*.csv'
GROUP BY Quality
ORDER BY Quality DESC;
```

Quality	Lignes	Ce que le code veut dire
192	275 080	bonne
64	35	douteuse
0	321	mauvaise, et la valeur est vide

Presque tout est bon, et c'est justement le problème : les 356 autres lignes se cachent dans la masse. On les traquera aux Cours 08 et 11.

Le nombre de points par étiquette :

```
-- Attendu : 58 lignes, ESS.BANC.TEMP_REFROIDISSEMENT en tête avec 27 661 points, UTL.CMP3.MARCHE en queue avec 210.
```

```
SELECT TagName, COUNT(*) AS points
FROM 'sources/usine/historien/*.csv'
GROUP BY TagName
ORDER BY points DESC;
```

DuckDB n'affiche pas les 58 lignes : il montre le haut et le bas du classement, remplace le milieu par des points, et dit sous le tableau combien de lignes il a montrées. En tête, ESS.BANC.TEMP_REFROIDISSEMENT, la température du liquide de refroidissement du véhicule à l'essai, avec 27 661 points : chaque véhicule terminé passe sept minutes au banc, et elle ne tient jamais en place. En queue, les états de marche (MARCHE) et les compteurs des convoyeurs, jusqu'à UTL.CMP3.MARCHE, le compresseur d'appoint, avec 210 points. Le SCADA lit pourtant chaque point toutes les deux minutes, soit 66 240 lectures en 92 jours. L'historien ne les garde pas toutes. Pour une mesure, il retient un point quand la valeur bouge de plus que sa bande morte, et au moins un point par heure ; un état ou un compteur, seulement à son changement. S'ajoutent, pour chaque étiquette, un point archivé de force à minuit et une valeur de départ en tête de chaque fichier.

Enfin, les premiers points d'une étiquette :

```
-- Attendu : 10 lignes, du 2025-09-30 23:36:00 au 2025-10-01 06:06:00.
```

```
SELECT Timestamp, Value, Quality
FROM 'sources/usine/historien/*.csv'
WHERE TagName = 'UTL.CMP2.VIBRATION'
```

```
ORDER BY Timestamp
LIMIT 10;
```

Le premier point date de la veille, 23 h 36 : c'est la valeur de départ, recopiée en tête du fichier du 1er octobre. Suit le point archivé de force à minuit, puis un point par heure pile jusqu'à 5 h : le compresseur est arrêté, sa vibration ne bouge pas, et seul le point obligatoire de l'historien s'écrit. À 6 h, le compresseur démarre, une demi-heure avant les lignes d'assemblage : la vibration passe à 2,44 mm/s, et les points se resserrent, deux minutes, puis quatre. Une série d'historien n'est pas une grille régulière, et c'est la première chose à comprendre avant de calculer quoi que ce soit dessus (Cours 08).

L'historien n'est qu'une des six sources. Un coup d'œil à l'ERP : les en-têtes de commande, la table VBAK, dans tous les extraits mensuels à la fois.

```
-- Attendu : 26 538 lignes, 25 483 commandes distinctes.
SELECT COUNT(*) AS lignes, COUNT(DISTINCT VBELN) AS commandes
FROM 'sources/erp/extraits/*/VBAK.csv';
```

Plus de lignes que de commandes : un extrait n'est pas une copie de la base. Chaque mois, l'ERP extrait les commandes créées ou modifiées dans le mois, et une commande modifiée le mois suivant revient dans l'extrait suivant. Et un extrait a été livré deux fois :

```
-- Attendu : 2 lignes, 609 en-têtes dans chacun.
SELECT '2023-03' AS extrait, COUNT(*) AS en_tetes FROM 'sources/erp/extraits/2023-03/VBAK.csv'
UNION ALL
SELECT '2023-03_bis', COUNT(*) FROM 'sources/erp/extraits/2023-03_bis/VBAK.csv';
```

Deux dossiers, les mêmes 609 commandes. Un pipeline qui charge tout ce qu'il trouve compterait les ventes de mars 2023 deux fois, sans la moindre erreur. Le Cours 03 en fait sa leçon d'idempotence.

Dernière source du jour, la télémétrie :

```
-- Attendu : 158 146.
SELECT COUNT(*) AS messages
FROM 'sources/telemetrie/*.jsonl';
```

DuckDB lit le JSON lines aussi simplement que le CSV : 158 146 messages, dont les 2 303 doublons qu'annonçait le bilan du générateur. Ils ne se voient pas encore ; le Cours 09 dit d'où ils viennent.

Reste à peser ce qui est arrivé. read_blob liste des fichiers avec leur taille :

```
-- Attendu : historien, 92 fichiers, 13 725 380 octets, aucun vide ;
-- télémétrie, 884 fichiers, 70 570 165 octets, dont 41 vides.
SELECT 'historien' AS source, COUNT(*) AS fichiers, SUM(size) AS octets,
COUNT(*) FILTER (WHERE size = 0) AS vides
FROM read_blob('sources/usine/historien/*.csv')
UNION ALL
SELECT 'telemetrie', COUNT(*), SUM(size), COUNT(*) FILTER (WHERE size = 0)
FROM read_blob('sources/telemetrie/*.jsonl');
```

13,7 Mo d'historien pour trois mois, 70,6 Mo de télémétrie pour deux ans et demi. Retenez ces deux nombres : ils serviront d'étalon au Cours 02, quand le bronze les rangera en Parquet. Quittez DuckDB avec `.exit`.

Piège — le chemin des fichiers est relatif au terminal. `'sources/usine/historien/*.csv'` se lit à partir du dossier où vous avez lancé duckdb. Si DuckDB répond `No files found that match the pattern`, vous n'êtes pas dans `Projet_Boreal`.

Étape 3 — Mesurer ce que coûte un type deviné

Théorie : Theorie_01.md, section 7.

Toujours dans `Projet_Boreal`, ouvrez la ligne de commande DuckDB (`duckdb`). D'abord les en-têtes de commande de l'ERP, VBAK, dans tous les extraits, tels que DuckDB les devine :

```
-- Attendu : 12 colonnes. MANDT bigint, VBELN varchar, ERDAT bigint, ERZET varchar, AEDAT varchar,  
-- AUART varchar, VKORG bigint, VTWEG bigint, KUNNR varchar, VDATU bigint, NETWR double, WAERK varchar.  
DESCRIBE SELECT * FROM 'sources/erp/extraits/*/VBAK.csv';
```

Les numéros qui portent des zéros de tête (VBELN, KUNNR) restent du texte : DuckDB sait qu'un entier les abîmerait. Le reste est moins heureux. ERDAT et VDATU, deux dates SAP, deviennent des entiers ; AEDAT, la même sorte de date, reste du texte, parce qu'elle vaut souvent 00000000. Les codes VKORG et VTWEG deviennent des nombres, et le montant NETWR, un double.

Le premier effet, sur la date de création :

```
-- Attendu : 20251231 et 20251232.  
SELECT MAX(ERDAT) AS dernière_creation, MAX(ERDAT) + 1 AS lendemain  
FROM 'sources/erp/extraits/*/VBAK.csv';
```

DuckDB calcule 20251232 sans broncher : pour lui, ERDAT est un nombre, et un nombre plus un donne le nombre suivant. Aucune erreur, un résultat faux. La bonne façon lit tout en texte, puis convertit avec le format connu :

```
-- Attendu : 2026-01-01.  
SELECT MAX(STRPTIME(ERDAT, '%Y%m%d'))::DATE + 1 AS lendemain  
FROM read_csv('sources/erp/extraits/*/VBAK.csv', all_varchar = true);
```

`all_varchar = true` coupe la détection : chaque colonne arrive en texte, et c'est vous qui décidez de sa conversion. Au Cours 03, la couche RAW gardera l'ERP ainsi, en texte.

La télémétrie, maintenant. L'heure de réception d'un message, telle que DuckDB la devine, puis telle qu'elle est écrite dans le fichier :

```
-- Attendu : 2025-12-15 05:25:22, de type TIMESTAMP.  
SELECT recu_le, TYPEOF(recu_le) AS type  
FROM 'sources/telemetry/sorties_2025-12-15.jsonl'  
LIMIT 1;
```

```
-- Attendu : 2025-12-15T00:25:22-05:00, de type VARCHAR.  
SELECT recu_le, TYPEOF(recu_le) AS type  
FROM read_json('sources/telemetrie/sorties_2025-12-15.jsonl', columns = {'recu_le': 'VARCHAR'})  
LIMIT 1;
```

Le message est arrivé à 0 h 25, heure de l’Est. DuckDB l’a ramené à 5 h 25 en UTC et a jeté le décalage -05:00. L’instant est juste, mais l’information du fuseau a disparu : pour debut et fin, c’était le fuseau où roulait le véhicule.

Dernier piège, celui qui plante :

```
-- Attendu : une seule colonne, json, de type json.  
DESCRIBE SELECT * FROM read_json('sources/telemetrie/*.jsonl');
```

Lue d’un seul motif, l’archive entière devient une colonne json. Les 39 premiers jours de l’archive, du 2023-08-01 au 2023-09-08, sont des fichiers vides : la détection, qui échantillonne les premiers fichiers, n’a rien trouvé à analyser. Une requête sur message.sennr échouerait ensuite, loin de sa cause.

La leçon tient en une règle, celle du bronze, que le Cours 02 applique : on déclare les types, on ne les devine pas, et l’on ne déclare un type que si la conversion ne perd rien. Quittez DuckDB avec .exit.

Récapitulatif

Vous avez...	Preuve
créé le dépôt	01_de/ est un dépôt Git ; .gitignore validé, git status propre
interrogé les sources en SQL	275 436 lignes d’historien comptées sans rien charger, dont 275 080 de qualité 192 ; 26 538 lignes d’en-têtes pour 25 483 commandes ; 158 146 messages de télémétrie
mesuré les sources	historien 13 725 380 octets en 92 fichiers ; télémétrie 70 570 165 octets en 884 fichiers, dont 41 vides
vu ce que coûte un type deviné	MAX(ERDAT) + 1 rend 20251232 ; recu_le perd son décalage ; l’archive entière devient une colonne json

Prochaine étape (Cours 02). Vous avez regardé sans rien ranger. Au Cours 02, le lac garde l’historien et la télémétrie en Parquet, avec des types déclarés et un dossier par jour livré, et l’on mesure ce que le format change, en octets et en secondes.



Dépannage

Symptôme	Cause probable	Correctif
DuckDB : No files found that match the pattern	duckdb lancé ailleurs que dans Projet_Boreal	quitter DuckDB (.exit), se placer dans le projet, relancer duckdb
Après Entrée, DuckDB n'affiche rien et attend la suite	la requête n'a pas son point-virgule final	taper ; , puis Entrée
Le décompte ne donne pas 275 436 lignes	le générateur a tourné avec --historien complet ou --jusqua, ou un fichier s'est ajouté dans historien/	relancer le générateur sans option, depuis Data_Domain : il refait sources/ de zéro
Binder Error: Referenced table "message" not found!	l'archive a été lue d'un seul motif, sans déclarer sa forme : une seule colonne json	déclarer les colonnes, ou ajouter union_by_name = true : le script du bronze fera les deux, au Labo 02

Annexe — Tout le code, d'un seul tenant

Pour rejouer le labo sans relire les explications. Les commandes de terminal se tapent dans l'Anaconda Prompt, depuis Projet_Boreal ; les requêtes, dans la ligne de commande DuckDB, lancée depuis le même dossier.

1 — Le dépôt (Anaconda Prompt ; le .gitignore dans le Bloc-notes)

```
mkdir 01_de
cd 01_de
git init
notepad .gitignore

.env
__pycache__/

git add .gitignore
git commit -m "Cours 01 : le dépôt 01_de et son .gitignore"
git status
cd ..
```

2 — Le premier regard et le poids des sources (DuckDB)

```
duckdb
```

Puis, à l'invite D :

```
SELECT COUNT(*) AS lignes FROM 'sources/usine/historien/*.csv';
DESCRIBE SELECT * FROM 'sources/usine/historien/*.csv';
SELECT Quality, COUNT(*) AS points FROM 'sources/usine/historien/*.csv' GROUP BY Quality ORDER BY Quality DESC;
SELECT TagName, COUNT(*) AS points FROM 'sources/usine/historien/*.csv' GROUP BY TagName ORDER BY points DESC;
SELECT Timestamp, Value, Quality FROM 'sources/usine/historien/*.csv' WHERE TagName = 'UTL.CMP2.VIBRATION' ORDER BY Timestamp
LIMIT 10;
SELECT COUNT(*) AS lignes, COUNT(DISTINCT VBELN) AS commandes FROM 'sources/erp/extraits/*/VBAK.csv';
SELECT '2023-03' AS extrait, COUNT(*) AS en_tetes FROM 'sources/erp/extraits/2023-03/VBAK.csv' UNION ALL SELECT '2023-03_bis',
COUNT(*) FROM 'sources/erp/extraits/2023-03_bis/VBAK.csv';
SELECT COUNT(*) AS messages FROM 'sources/telemetrie/*.jsonl';
SELECT 'historien' AS source, COUNT(*) AS fichiers, SUM(size) AS octets, COUNT(*) FILTER (WHERE size = 0) AS vides FROM
read_blob('sources/usine/historien/*.csv') UNION ALL SELECT 'telemetrie', COUNT(*), SUM(size), COUNT(*) FILTER (WHERE size = 0)
FROM read_blob('sources/telemetrie/*.jsonl');
.exit
```

3 — Les types devinés (DuckDB)

duckdb

Puis, à l'invite D :

```
DESCRIBE SELECT * FROM 'sources/erp/extraits/*/VBAK.csv';
SELECT MAX(ERDAT) AS derniere_creation, MAX(ERDAT) + 1 AS lendemain FROM 'sources/erp/extraits/*/VBAK.csv';
SELECT MAX(STRPTIME(ERDAT, '%Y%m%d'))::DATE + 1 AS lendemain FROM read_csv('sources/erp/extraits/*/VBAK.csv', all_varchar =
true);
SELECT recu_le, TYPEOF(recu_le) AS type FROM 'sources/telemetrie/sorties_2025-12-15.jsonl' LIMIT 1;
SELECT recu_le, TYPEOF(recu_le) AS type FROM read_json('sources/telemetrie/sorties_2025-12-15.jsonl', columns = {'recu_le':
'VARCHAR'}) LIMIT 1;
DESCRIBE SELECT * FROM read_json('sources/telemetrie/*.jsonl');
.exit
```